R1.01 - Initiation au Développement : Variables, Mémoire RAM, Structures Conditionnelles et Répétitives

Thème principal : Bases de l'algorithmique impérative, gestion mémoire et structures de contrôle

Synthèse exécutive : Ce cours pose les fondations absolues de la programmation informatique pour les étudiants en BUT Informatique à l'IUT de Bayonne et du Pays Basque. Il détaille l'interaction entre le code source et la mémoire vive (RAM) à travers le typage et l'affectation destructive des variables. Il formalise également l'aiguillage du flux séquentiel via les branchements conditionnels (SI / ALORS / SINON) fondés sur l'algèbre de Boole, ainsi que l'automatisation des traitements itératifs par les boucles déterministes (POUR) et conditionnelles (TANT QUE), avec un focus majeur sur la prévention des boucles infinies et le choix rigoureux des structures.

Les Variables, le Typage et l'Allocation en Mémoire RAM

En programmation impérative, le processeur manipule des données conservées temporairement dans la mémoire vive (RAM). Pour éviter de gérer directement des adresses hexadécimales physiques complexes (ex: 0x7FFF5FBFF058), l'algorithmique introduit la notion de variable. Toute variable doit être explicitement déclarée avant son utilisation. La déclaration informe l'ordinateur qu'il doit réserver un espace mémoire spécifique en lui attribuant un identificateur (son nom) et un type. L'opération d'affectation respecte un ordre chronologique strict : 1. L'expression située à droite de l'opérateur est intégralement calculée par le processeur ; 2. Le résultat est stocké dans la case mémoire associée à la variable située à gauche. Cette écriture est dite destructive car la valeur qui s'y trouvait préalablement est définitivement effacée et irrécupérable.

- Variable : Emplacement mémoire nommé (adresse abstraite en RAM) permettant de conserver et modifier une valeur pendant l'exécution d'un programme.
- Typage : Spécification du format de donnée (Entier, Réel, Chaîne, Booléen) fixant sa taille réservée en octets (ex: 4 octets pour un entier 32 bits, 8 octets pour un double) et les opérations autorisées.
- Affectation (notée <- ou =) : Opération séquentielle et destructive évaluant d'abord l'expression située à droite puis écrivant le résultat dans la variable située à gauche.

Cas concret enseignant : *Le professeur compare la mémoire RAM à un grand meuble à tiroirs : déclarer la variable 'age : Entier', c'est coller une étiquette 'age' sur un tiroir spécifique et fabriquer physiquement ce tiroir à la dimension exacte pour recevoir uniquement un nombre entier. Placer une nouvelle valeur dans ce tiroir chasse et détruit instantanément la valeur précédente.*

Les Structures Conditionnelles et l'Algèbre de Boole

Par défaut, un programme informatique s'exécute de façon linéaire et séquentielle, ligne par ligne. Les structures conditionnelles rompent cette séquentualité en offrant des branchements d'exécution alternatifs selon le résultat d'une évaluation booléenne. Le processeur évalue d'abord la condition : si elle renvoie VRAI, les instructions du bloc 'ALORS' sont exécutées et le bloc 'SINON' est ignoré. Si la condition renvoie FAUX, le processeur saute directement au bloc 'SINON' (s'il existe) ou poursuit la suite du programme. En partiel, une attention particulière doit être apportée à l'exhaustivité des cas pour s'assurer qu'aucun état limite du système ne reste non géré.

- Condition Booléenne : Expression logique s'évaluant exclusivement à VRAI (True) ou FAUX (False).
- Bloc SI / ALORS / SINON : Structure de contrôle permettant d'aiguiller le flux d'instructions selon le résultat d'un test booléen.
- Opérateurs Logiques (ET, OU, NON) : Connecteurs permettant de combiner plusieurs conditions simples afin d'exprimer des règles de décision complexes.

Cas concret enseignant : *Exemple du contrôle d'accès au parking du campus d'Anglet / Bayonne : Si (carte_etudiant_valide == VRAI ET parking_complet == FAUX) ALORS lever la barrière ; SINON afficher le message d'erreur 'Accès Refusé' sur l'écran.*

Les Structures Répétitives (Boucles POUR et TANT QUE)

Les structures répétitives ou itératives permettent de répéter un même bloc d'instructions sans dupliquer le code source. Le choix entre une boucle POUR et une boucle TANT QUE relève d'une décision d'architecture algorithmique fondamentale. La boucle POUR est préconisée lorsque le nombre d'itérations est déterminé au départ (par exemple parcourir les 500 étudiants d'une promotion). La variable de contrôle s'incrémente automatiquement à chaque passage. La boucle TANT QUE est utilisée lorsque la répétition dépend d'une condition dynamique (par exemple attendre une saisie valide). La condition est évaluée AVANT chaque passage : si elle est fausse dès le premier test, la boucle s'exécute 0 fois. Règle d'or absolue d'ingénierie : dans un TANT QUE, au moins une variable engagée dans la condition de test doit obligatoirement être modifiée à l'intérieur du corps de la boucle pour garantir l'arrêt du programme.

- Boucle POUR (FOR) : Structure itérative déterministe avec compteur automatique, réservée aux cas où le nombre d'itérations est fixe et connu avant le début de la boucle.
- Boucle TANT QUE (WHILE) : Structure itérative conditionnelle, exécutée tant qu'une condition logique reste vraie, adaptée aux répétitions indéterminées.
- Boucle Infinie : Dysfonctionnement critique survenant lorsque la condition de sortie d'une boucle TANT QUE ne devient jamais fausse, bloquant indéfiniment le programme.

Cas concret enseignant : *Le bug du distributeur automatique de billets (DAB) : si l'algorithme oublie de remettre la variable 'carte_inseree' à FAUX après l'éjection physique de la carte, la boucle TANT QUE demandant la saisie du code PIN tourne indéfiniment et bloque l'appareil.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi doit-on obligatoirement spécifier le type d'une variable dans des langages comme C ou C++ alors que ce n'est pas nécessaire en Python ?
Réponse du professeur : Dans les langages typés statiquement et compilés comme C ou C++, la spécification du type permet au compilateur de réserver exactement l'espace mémoire nécessaire en RAM (ex: 4 octets pour int) avant l'exécution. En Python, le typage est dynamique : l'interprète enveloppe la donnée dans un objet complexe qui transporte son type et effectue la gestion mémoire au moment de l'exécution, ce qui consomme plus de mémoire et ralentit l'exécution.

Question étudiant : Quelle est la différence exacte entre l'opérateur '=' et l'opérateur '==' ?
Réponse du professeur : L'opérateur '=' est une instruction d'affectation (action destructive qui évalue la partie droite et stocke le résultat dans la variable à gauche). L'opérateur '==' est un test de comparaison d'égalité (opération d'évaluation logique qui renvoie un booléen VRAI ou FAUX sans modifier le contenu des variables).

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme C++ complet respectant les normes de l'IUT d'Anglet qui demande 5 notes (sur 20) à l'utilisateur, calcule leur moyenne arithmétique, puis affiche 'Admis' si la moyenne est supérieure ou égale à 10.0, ou 'Ajourné' dans le cas contraire.

Démarche de résolution :
1. Structuration du fichier C++ avec cartouche complet, tripartition (CONSTANTES, VARIABLES, TRAITEMENTS) et indentation Allman. 2. Définition des constantes NB_NOTES = 5 et SEUIL_ADMISSION = 10.0. 3. Initialisation de l'accumulateur 'sommeNotes' à 0.0 avant d'entrer dans la boucle. 4. Utilisation d'une boucle POUR (for) de 1 à 5 car le nombre de répétitions est déterministe. 5. Lecture de chaque note via le flux 'cin' et ajout immédiat dans 'sommeNotes'. 6. Calcul de la moyenne par division réelle par la constante NB_NOTES. 7. Application du test conditionnel 'if (moyenneCalculee >= SEUIL_ADMISSION)' pour déterminer l'affichage final.

```cpp
//============================================================================
// NOM       : TP_CalculMoyenne.cpp
// AUTEUR    : Matéi de Saint Leger
// CLASSE    : BUT1 Informatique - IUT de Bayonne et du Pays Basque (Anglet)
// MATIERE   : R1.01 - Initiation au Développement
// OBJECTIF  : Saisie de 5 notes, calcul de moyenne et décision d'admission
//============================================================================

#include <iostream>
using namespace std;

int main()
{
    //---------------------------------------------------------------------
    // CONSTANTES
    //---------------------------------------------------------------------
    const int NB_NOTES = 5;
    const double SEUIL_ADMISSION = 10.0;

    //---------------------------------------------------------------------
    // VARIABLES
    //---------------------------------------------------------------------
    double noteCourante = 0.0;
    double sommeNotes = 0.0;
    double moyenneCalculee = 0.0;
    int i = 0;

    //---------------------------------------------------------------------
    // TRAITEMENTS
    //---------------------------------------------------------------------
    for (i = 1; i <= NB_NOTES; i++)
    {
        cout << "Veuillez saisir la note n°" << i << " (sur 20) : ";
        // (clavier) >> saisir >> noteCourante
        cin >> noteCourante;
        sommeNotes = sommeNotes + noteCourante;
    }

    moyenneCalculee = sommeNotes / NB_NOTES;

    cout << "\nMoyenne calculée : " << moyenneCalculee << " / 20" << endl;

    if (moyenneCalculee >= SEUIL_ADMISSION)
    {
        cout << "Résultat de l'étudiant : Admis" << endl;
    }
    else
    {
        cout << "Résultat de l'étudiant : Ajourné" << endl;
    }

    return 0;
}
```

Résultat : Le programme exécute exactement 5 itérations de façon séquentielle, cumule les saisies utilisateur de manière déterministe sans réinitialisation intempestive de la somme, effectue un calcul précis en virgule flottante et délivre le statut exact d'admission.

■ Auto-Évaluation Flash d'Examen

Q : Que se passe-t-il si la condition d'une boucle TANT QUE est évaluée à FAUX dès le premier contrôle ?

R : Le corps de la boucle TANT QUE n'est jamais exécuté (0 itération), et le programme passe immédiatement à la première instruction située après la boucle.

Q : Quelle est la règle d'or pour choisir entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR est obligatoire lorsque le nombre d'itérations est connu d'avance. La boucle TANT QUE s'impose lorsque le nombre de répétitions dépend d'une condition dynamique évaluée pendant l'exécution.

Q : Pourquoi l'opération d'affectation est-elle qualifiée de 'destructive' ?

R : Parce qu'une fois l'expression de droite évaluée, son résultat vient s'écrire dans la case mémoire de la variable à gauche, ce qui efface et détruit définitivement la valeur précédente.